

CSE-355: Introduction to Blockchain

Programming Assignment # 2

Deadline: 11:55 pm on 3rd May 2026

1 Introduction

XYZ, the biggest bank in Pakistan and a household name for decades, suddenly found itself at a crossroads. Traditional banking systems were starting to feel a little too old-school. Endless paperwork, slow transaction settlements, and the ever-present headache of middlemen meant customers were getting restless. Meanwhile, executives started hearing about a new innovation in financial technologies: Blockchain.

At first, the executives at XYZ chuckled that this is useless. But soon, after seeing the global shift in finance, the board realized: blockchain wasn't a fad. It was the future.

XYZ Bank has embarked on a bold reinvention of its operations. The vision is clear: to migrate core banking processes onto blockchain technology in order to eliminate inefficiencies, enhance transparency, and deliver faster, more reliable services to customers. With smart contracts at the foundation, transactions become instantaneous, auditable, and secure—removing unnecessary delays, hidden fees, and the uncertainty of pending approvals. This marks a shift toward a more open, efficient, and customer-centric model of banking.

But here's the twist: you, a fresh graduate from IBA who had just taken the Intro to Blockchain course, spotted this golden opportunity. Why wait for the slow-moving bureaucracy of a big bank to figure it out when you could launch your own smart-contract based banking startup?

Armed with enthusiasm, caffeine, and the trusty Remix IDE (the favorite playground of beginner blockchain developers everywhere), you rolled up your sleeves. This was it—the first step in reimagining banking for Pakistan. And unlike XYZ's decades of paperwork and bureaucracy, your startup was going to run on pure code.

2 Remix IDE Setup

2.1 Skeleton Code

Download the **PA2_Skeleton.zip** file from the LMS. The archive contains the following components:

1. **Skeleton Smart Contract:** A template file named **banking_2xxxxxxx_contract.sol**, which you must complete as part of the assignment.
2. **Solidity Test Files:** Four pre-written test contracts (**banking_test.sol**, **exceptions_test.sol**, **interest_test.sol**, and **owner_test.sol**) designed for use within the Remix IDE.

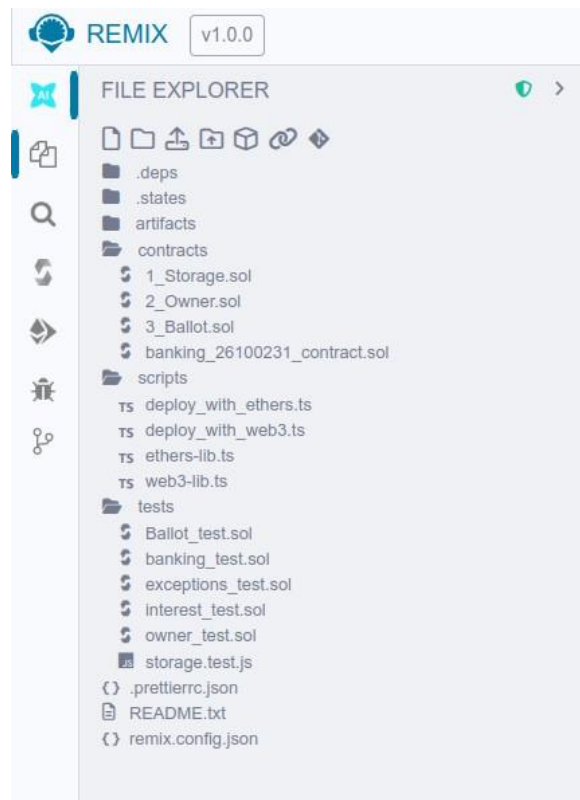
3. **JavaScript Test Files (Hardhat):** Four equivalent JavaScript test files (**banking_test.js**, **exceptions_test.js**, **interest_test.js**, and **owner_test.js**) provided for automated grading with Hardhat.

Before starting, rename your smart contract file according to your campus ID. For example: **banking_26100231_contract.sol**

2.2 Uploading files to Remix IDE

1. Navigate to <https://remix.ethereum.org/>.
2. On the home page, click on *Open files* icon and upload the **banking_2xxxxxxx_contract.sol**, and move it to the **contracts** directory.
3. From the **Solidity Test Files** folder downloaded from LMS, upload the 4 test files named **banking_test.sol**, **interest_test.sol**, **owner_test.sol**, **exceptions_test.sol** to remix IDE, and move them to the **tests** directory.
4. **IMPORTANT:** In each of the 4 solidity test files, update the import statement to the filename of your contract, like `import "../contracts/banking_26100231_contract.sol";`

Once done, the file explorer would look as follows:



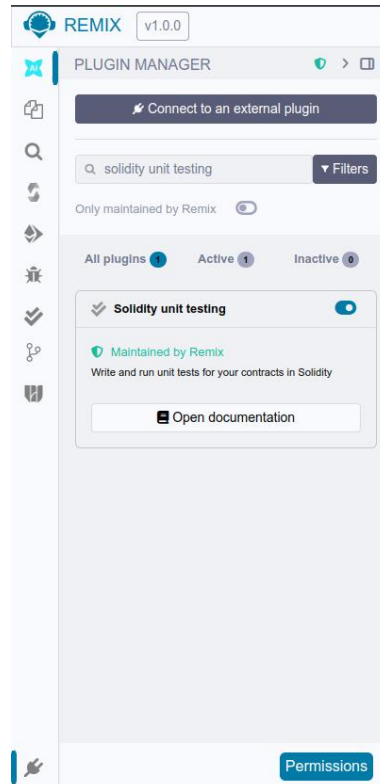
2.3 Activating the Solidity Unit Testing Plugin

1. Click on the plugin manager icon at the bottom left.

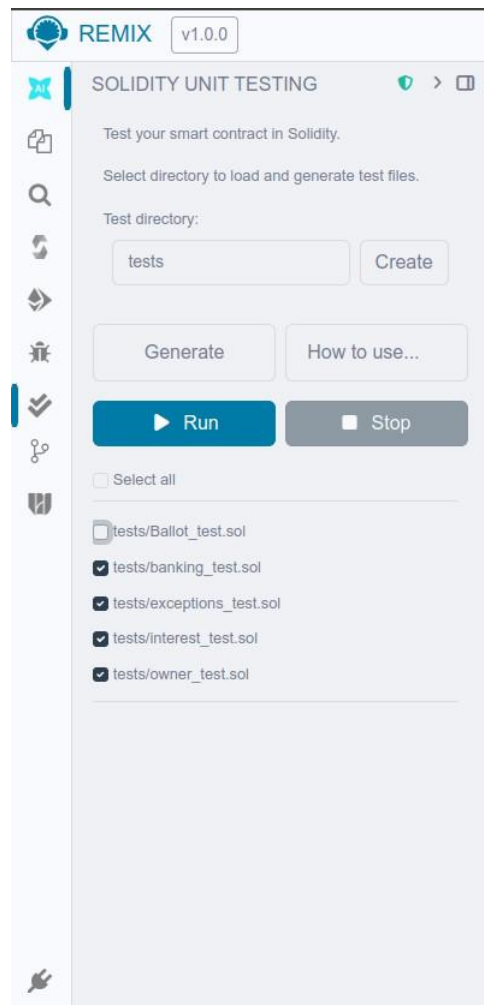


2. Search **Solidity unit testing** in the search bar

3. Click the toggle button to activate it



4. To run the test cases, click on the **double tick icon** in the side panel, select the test files to run and click **Run**.



3 Basic Structure of the Banking smart contract

The Remix Virtual Machine is an environment that simulates Ethereum Virtual Machine (EVM) on the browser, so that we can deploy and run smart contracts locally on our machine instead of on a real blockchain network. Remix gives us 15 dummy accounts, each owning 100 ETH for our use.

3.1 Basic Flow

1. The contract is deployed on the Remix VM by one account. This account becomes the **owner**, representing the bank itself.
2. All other accounts act as **customers**, who interact with the smart contract just as they would with a traditional bank.
3. Once deployed, the smart contract is assigned an address with an initial balance of 0 ETH. It functions as the “bank’s vault,” and its behavior is governed entirely by the code (which is immutable). The contract’s state can only be modified through its defined functions.
4. Customers interact with the bank by sending transactions that call the smart contract’s functions, similar to submitting requests at a physical bank.
5. For example, when a customer deposits funds, they call the deposit function and transfer ETH from their wallet to the contract. The smart contract (the bank) records this in its internal ledger.

3.2 Contract State (Global Variables)

The smart contract maintains several global state variables that define its operation. These are:

1. **owner**: The address of the contract owner. This represents the bank itself, responsible for managing system-level operations such as setting interest rates, topping up loan or operational funds, and applying interest.
2. **accounts**: A mapping from each user's address to an Account struct. This serves as the internal ledger of the bank.

Account: A custom defined struct that contains the following fields:

- **firstName** (string) – The customer's first name.
 - **lastName** (string) – The customer's last name.
 - **balance** (uint) – The customer's account balance.
 - **principalLoan** (uint) – The principal amount of the outstanding loans taken by the customer.
 - **interestLoan** (uint) – The outstanding interest amount on the loan to be paid by the customer.
 - **exists** (bool) – A flag indicating whether the account has been created.
3. **AccountKeys**: An array of all registered customer addresses. This provides a way to iterate through the list of customers.
 4. **depositInterestRate**: The interest rate (in percentage) applied to customer deposits.
 5. **loanInterestRate**: The interest rate (in percentage) applied to outstanding customer loans.
 6. **loan_funds**: The loan reserve maintained by the bank. This represents the ETH deposited by the owner for issuing loans. If the reserve is zero, new loans cannot be issued.
 7. **operational_funds**: The operational funds maintained by the bank for covering internal expenses. These can be topped up by the owner by depositing ETH. In this assignment we will be mainly using these funds for giving interests on customer deposits. The interest earned by the bank on the loans would be stored here.

3.2 Access Control (Modifiers)

To simplify repeated checks, you are expected to implement the following modifiers:

1. **onlyOwner**
Ensures that only the contract owner can call a function.
 2. **notOwner**
Prevents the owner from calling certain customer-only functions.
 3. **hasAccount**
Ensures that the caller has already created an account.
- You are expected to use these modifiers in relevant functions instead of rewriting the same checks repeatedly.

4 Compiling, Deploying and running smart contract

4.1 Compile

Click the **Solidity Compiler icon** in the side panel, and click Compile

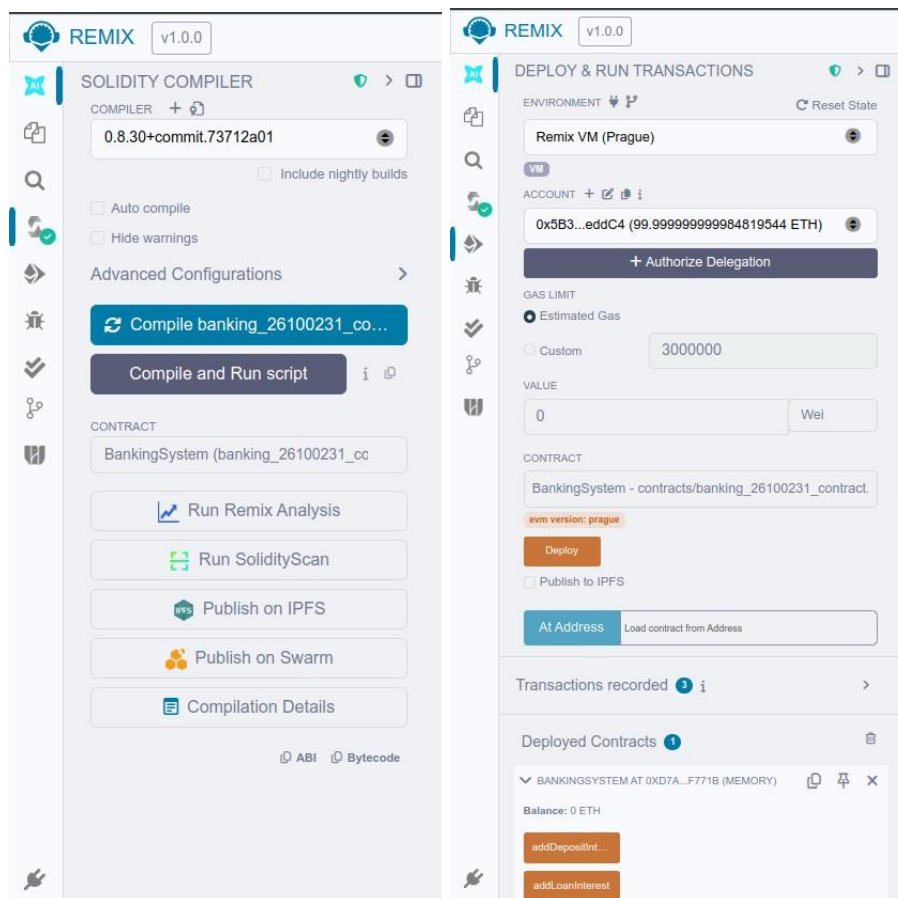
4.2 Deploy

1. Click on the **Deploy and run transactions** tab in the side panel
2. From the list of accounts, select the account which you want to make the owner of the contract.
3. Click **Deploy**

4.3 Calling Functions

1. After deploying the contract, the deployed contract instance will appear under the **Deploy and Run Transactions** window in Remix. All available functions of the contract are listed there.
2. You can select any account from the **Account** dropdown and call a function as that entity (either the owner, representing the bank, or a customer).
3. For payable (see Section 10) functions, you may also enter the amount of ETH to send along with the function call in the **Value** field to sent to smart contract (bank).

IMPORTANT: Every time you modify the smart contract code, you must recompile and redeploy it for changes to take effect. While you can enable **Auto Compile** in the Solidity Compiler tab, redeployment still needs to be done manually.



5 Implementation overview

5.1 List of functions

The following table summarizes the functions and the entities allowed to call them:

Function	Caller	Description	Payable
openAccount	Customer	Creates a new account in the bank.	No
getDetails	Customer	Retrieves the caller's account details.	No
depositAmount	Customer	Deposits ETH into caller's account.	Yes
withDraw	Customer	Withdraws funds from caller's account.	No
transferEth	Customer	Transfers funds to another customer.	No
depositTopUp	Owner (Bank)	Tops up loan reserve for issuing loans.	Yes
depositOperationalFunds	Owner (Bank)	Adds ETH to operational funds for bank use.	Yes
TakeLoan	Customer	Requests a loan from the bank.	No
InquireLoan	Customer	Checks current outstanding loan amount.	No
returnLoan	Customer	Repays outstanding loan amount (with ETH).	Yes
setInterestRates	Owner (Bank)	Updates deposit and loan interest rates.	No
addDepositInterest	Owner (Bank)	Adds interest to all customer deposits.	No
addLoanInterest	Owner (Bank)	Applies loan interest to borrowers.	No
closeAccount	Customer	Closes caller's account after ensuring balance is zero and no outstanding loan exists.	No
AmountInBank	Both	Returns the contract's ETH balance.	No
OperationalFunds	Both	Returns the current operational funds available.	No
DepositInterestRate	Both	Returns current deposit interest rate.	No
LoanInterestRate	Both	Returns current loan interest rate.	No
LoanFunds	Both	Returns available loan funds reserve.	No

The functions that are going to be implemented by students are defined in the following format in the manual:

- **Description** A short description of what the function does.
- **Inputs** – The arguments of the functions and their data type.
- **State Updates** – Changes to the contracts state (variables) after a particular function is executed.
- **Exceptions** – The exceptions that should be raised if a condition is not met.
- **Outputs** – The outputs returned by the function if any.

5.2 Constructor

You will be implementing the constructor of the banking smart contract firstly. The constructor is executed only once at the time of deployment and sets up the initial state of the contract.

1. Set the deployer of the contract as the **owner**.
2. Initialize the **deposit interest rate** and **loan interest rate** to zero.
3. Initialize the **loan reserve** and **operational funds** to zero.

Note:

1. The owner is the special address representing the bank itself. This account is prohibited from opening a user account.
2. All other functions will rely on the owner being set correctly here.
3. The constructor does not take any arguments and does not return any value.

6 Functions

6.1 Opening An Account

Function:

```
function openAccount(string memory firstName, string memory lastName) public
```

6.1.1 Description

Creates a new bank account for the caller with zero initial balance and no loan.

6.1.2 Inputs

1. firstName: string
2. lastName: string

6.1.3 State Updates

Store caller's address, firstName, lastName, and initialize balance, loanPrincipal and loanInterest to 0.
Add the caller's address to addressList

6.1.4 Exceptions

1. Account with the same user address already exists → "Account already exists"
2. Contract owner calls the function → "Error, Owner Prohibited"

6.1.5 Output

No return value.

IMPORTANT:

Whenever a new account is created, its address must be added to addressList.
This list will later be used for iterating over all users when applying interest.

6.2 Check account information

```
function getDetails() public view returns (uint, string memory, string memory, uint, uint)
```

6.2.1 Description

Retrieves account details for the caller.

6.2.2 Inputs

None (caller's address is used). (see Section 10)

6.2.3 State Updates

None (read-only).

6.2.4 Exceptions

1. Function caller does not have an account → "No Account"

6.2.5 Output

Returns: (uint balance, string memory firstName, string memory lastName, uint loanPrincipal, uint loanInterest).

6.3 Depositing Ether

Function:

```
function depositAmount() public payable
```

6.3.1 Description

Allows customers to deposit Ether into their bank account.

6.3.2 Inputs

None (msg.value is used). (see Section 10)

6.3.3 State Updates

Increase caller's balance by msg.value. Ether is stored in the contract.

6.3.4 Exceptions

1. Deposit < 1 ether → "Low Deposit"
2. Function caller does not have an account → "No Account"

6.3.5 Output

No return value.

6.4 Withdrawing Ether

Function:

```
function withdraw(uint withdrawalAmount) public
```

6.4.1 Description

Withdraws Ether from the caller's bank account.

6.4.2 Inputs

1. withdrawalAmount: uint

6.4.3 State Updates

Decrease caller's balance and transfer Ether to the caller.

6.4.4 Exceptions

1. Contract owner calls the function → "Error, Owner Prohibited"
2. Function caller does not have an account → "No Account"
3. Withdrawal exceeds available balance → "Insufficient Funds"

6.4.5 Output

No return value. Only the ethers are transferred to the customer. (see Section 10)

6.5 Transferring Ether

Function:

```
function TransferEth(address recipient, uint transferAmount) public
```

6.5.1 Description

Transfers Ether from the caller's account to another account within the bank.

6.5.2 Inputs

1. recipient: address payable (see Section 10)
2. transferAmount: uint

6.5.3 State Updates

Decrease sender balance, increase recipient balance.

6.5.4 Exceptions

1. Contract owner calls the function → "Error, Owner Prohibited"
2. Function caller does not have an account → "No Account"
3. Recipient does not have an account → "Recipient account does not exist"
4. Transfer exceeds available balance → "Insufficient Funds"

6.5.5 Output

No return value.

6.6 Deposit Top Up (Loan Reserve)

Function:

```
function depositTopUp() public payable
```

6.6.1 Description

Allows the owner to add funds to the loan reserve.

6.6.2 Inputs

None (msg.value is used). (see Section 10)

6.6.3 State Updates

Increase loan reserve.

6.6.4 Exceptions

1. Only contract owner can call this function → "Only Owner can call this function"

6.6.5 Output

No return value.

6.7 Deposit Operational Funds

Function:

```
function depositOperationalFunds() public payable
```

6.7.1 Description

Allows the owner to add funds to the operational funds reserve.

6.7.2 Inputs

None (msg.value is used). (see Section 10)

6.7.3 State Updates

Increase operational_funds by the amount transferred.

6.7.4 Exceptions

1. Only contract owner can call this function → "Only Owner can call this function"

6.7.5 Output

No return value.

6.8 Taking a Loan

Function:

```
function TakeLoan(uint loanAmount) public
```

6.8.1 Description

Allows customers to borrow funds from the bank's loan reserve.

6.8.2 Inputs

1. loanAmount: uint

6.8.3 State Updates

Decrease loan funds reserve, increase caller's loanPrincipalAmount, transfer ETH to customer's wallet. (see Section 10)

6.8.4 Exceptions

1. Contract owner calls the function → "Error, Owner Prohibited"
2. Function caller does not have an account → "No Account"
3. Loan exceeds available loan reserve → "Insufficient Loan Funds"
4. Loan amount exceeds twice the user's balance → "Loan Limit Exceeded"

6.8.5 Output

No return value. Only the ethers are transferred to the customer. (see Section 10)

6.9 Inquire Loan

Function:

function InquireLoan() public view returns (uint, uint, uint)

6.9.1 Description

Returns the outstanding loan principal amount, interest amount and their total.

6.9.2 Inputs

None (caller's address is used).

6.9.3 State Updates

None.

6.9.4 Exceptions

1. Function caller does not have an account → "No Account"

6.9.5 Output

Returns: (uint loanPrincipalAmount, uint loanInterestAmount, uint total)

6.10 Returning a Loan

Function:

```
function returnLoan() public payable
```

6.10.1 Description

Allows a user to repay their loan.

6.10.2 Inputs

None (msg.value is used). (see Section 10)

6.10.3 State Updates

If the payment is less than or equal to loanInterestAmount, decrease the loanInterestAmount, and then increase the operational_funds by the paid amount.

If the payment is more than the loanInterestAmount, decrease the loanInterestAmount, and then increase the operational_funds by the paid amount. Then decrease the loanPrincipalAmount by the remaining amount, and increase the loan_funds by that amount.

We are assuming that the loan and interest may be paid partially in installments.

6.10.4 Exceptions

1. No loan → "No Loan"
2. Function caller does not have an account → "No Account"
3. msg.value > loanPrincipalAmount+loanInterestAmount → "Owed Amount Exceeded"

6.10.5 Output

No return value.

6.11 Set Interest Rates

Function:

```
function setInterestRates(uint depRate, uint loanRate) public
```

6.11.1 Description

Allows the owner to update deposit and loan interest rates.

6.11.2 Inputs

1. depRate: uint
2. loanRate: uint

6.11.3 State Updates

Update deposit and loan interest rates.

6.11.4 Exceptions

1. Only contract owner can call the function → "Only the owner can set interest rates"

6.11.5 Output

No return value.

6.12 Add Deposit Interest

Function:

```
function addDepositInterest() public
```

6.12.1 Description

Owner adds deposit interest to all customers' balances.

6.12.2 Inputs

None.

6.12.3 State Updates

For each account: Add $(\text{balance} \times \text{depositInterestRate} / 100)$ to the existing balance

6.12.4 Exceptions

1. Only owner can call the function → "Only the owner can add interest to deposits"
2. If the sum of all deposit interests to be given is more than operational_funds, then revert the transaction with the error message "Not enough operational funds to pay interest"

6.12.5 Output

No return value.

6.13 Add Loan Interest

Function:

```
function addLoanInterest() public
```

6.13.1 Description

Owner adds loan interest to all customers' outstanding loans.

6.13.2 Inputs

None.

6.13.3 State Updates

For each account: add $\text{loan} \times \text{loanInterestRate} / 100$.

6.13.4 Exceptions

1. Only owner can call the function → "Only the owner can add interest to loans"

6.13.5 Output

No return value.

6.14 Closing an Account

Function:

```
function closeAccount() public
```

6.14.1 Description

Allows a user to close their bank account.

6.14.2 Inputs

None.

6.14.3 State Updates

Remove account details from storage.

6.14.4 Exceptions

1. Contract owner calls the function → "Error, Owner does not own an account"
2. No user account exists → "No Account Exists"
3. Loan outstanding → "Dues remaining, cannot close account before repayment"
4. If the customer balance > 0 → "Outstanding balance, withdraw it to close your account"

6.14.5 Output

No return value.

7 Grading

We will be using Hardhat for grading. Hardhat is an ethereum development framework used for compiling, deploying, testing, and debugging Solidity smart contracts in professional settings.

We have provided you with 4 JavaScript test files (**banking_test.js**, **exceptions_test.js**, **interest_test.js**, and **owner_test.js**) (equivalent to the solidity test files used in remix). If the test cases in remix were passing in your contract, they would also pass here.

NodeJS is a pre-requisite to run hardhat, so install nodeJS if you haven't already.

Follow these steps to setup Hardhat and run these files:

1. In a **new empty directory**, run `npm init -y`
2. Then run `npm install --save-dev hardhat@2.26.0`
3. After hardhat is installed, run `npx hardhat@2 init`

10 Appendix

10.1 Payable Functions and the msg Object

10.1.1 What Is a payable Function?

In Solidity, a function declared with the payable modifier is allowed to receive Ether along with the call. If a function is *not* payable, then sending Ether (i.e. nonzero msg.value) to it will cause the transaction to revert.

10.1.2 The msg Global Object

When a function is invoked on a smart contract, Solidity provides a global msg object that carries metadata about the transaction. The fields of interest include:

- msg.sender (address): the address initiating the call (the immediate caller).
- msg.value (uint): the amount of Wei (Ether) sent along with the call.
- msg.data (bytes calldata): the raw input data to the function.
- msg.sig (bytes4): the first four bytes of msg.data, representing the function selector.

The most frequently used fields in our assignment are msg.sender (to identify who called the function) and msg.value (to access how much Ether was sent).

10.1.3 Example 1: Receiving Ether

A contract can accept Ether through a payable function. For example:

```
function storeEther() public payable {
    require(msg.value > 0, "Must send some Ether");
    // Ether sent is automatically added to contract's balance
}
```

Here, msg.value contains the amount of Ether provided in the call, and the contract's balance increases accordingly.

10.1.4 Example 2: Transferring Ether Out

A contract can also send Ether to an external address:

```
function sendEther(address payable recipient, uint amount) public {
    require(address(this).balance >= amount, "Insufficient contract balance");
    (bool success, ) = recipient.call{value: amount}("");
    require(success, "Transfer failed");
}
```

The ethers are transferred to the address associated with the payable keyword in function signature. This shows how a contract uses the low-level call method to transfer Ether safely. The success flag ensures the transfer does not silently fail.

10.2 Transaction Fields in Solidity

10.2.1 The tx Object

In Solidity, the global tx object provides information about the entire transaction rather than just the current call. Its most notable field is:

- tx.origin (address): the address of the externally owned account (EOA) that originally started the transaction. Unlike msg.sender, this value does not change even if the transaction passes through multiple contracts.

10.2.2 Why It Matters in Hardhat Testing

When testing with the Hardhat framework, all transactions are initiated from EOAs (accounts provided by Hardhat). This means:

- msg.sender reflects the *immediate caller* (either an EOA or another contract in the call chain).
- tx.origin always reflects the original EOA that triggered the transaction from the Hardhat test script.

This distinction helps demonstrate how execution context changes as a transaction propagates through multiple contracts, even in a local testing setup.

10.2.3 Example: Nested Calls

```
contract A {
    function callB(address bAddr) public returns (address, address) {
        B b = B(bAddr);
        return b.getCallers();
    }
}

contract B {
    function getCallers() public view returns (address, address) {
        return (msg.sender, tx.origin);
    }
}
```

If Alice (an EOA provided by Hardhat) calls A.callB(), then inside contract B:

- msg.sender = address of contract A (the direct caller).
- tx.origin = Alice's address (the original transaction sender).

10.2.4 Summary

- The tx object exposes information about the overall transaction.
- In Hardhat, this makes tx.origin a stable way to observe the original EOA across contract calls, **hence you must use tx.origin in this assignment instead of msg.sender**
- However, in real smart contracts, msg.sender should be used for authorization and fund transfers. Using tx.origin in production is discouraged, as it can create security vulnerabilities (e.g., phishing through malicious intermediate contracts).

Good Luck!

